TripStack Flight Booking Automation Framework Report

Executive Summary

Developed an end-to-end UI automation framework for the TripStack flight booking application using Playwright and TypeScript. The framework follows a layered architecture focused on maintainability, reusability, scalability, and CI/CD readiness while incorporating secure test execution, structured logging, and evidence collection.

Technology Stack

- **Automation Tool:** Playwright
- **Language:** TypeScript
- **Test Runner:** Playwright Test
- **Design Pattern:** Flow + Page Object Model (POM)
- **Logging:** Winston
- **Reporting:** Playwright HTML Reports
- **CI/CD:** GitHub Actions
- **Configuration:** Environment Variables & GitHub Secrets

Framework Architecture

tests/

fixtures/

flows/

pages/

components/

support/

utils/

Layer	Responsibility
Tests	Business scenarios
Flows	Workflow orchestration and assertions

Layer	Responsibility
Pages	Locators and UI actions
Components	Reusable UI elements
Fixtures	Dependency injection and setup
Utilities	Logging, evidence, helpers

Automated Business Flow

Flight Search

- Navigate to Home Page
- Verify Flights tab is active
- Enter origin and destination
- Select travel date
- Execute flight search

Flight Listing

- Validate flight results are displayed
- Compare displayed result count against rendered flight cards
- Apply airline, time, price, and sorting filters
- Validate filtered results

Flight Details

- Open flight details page
- Select seat
- Verify seat selection

Passenger Details

- Enter passenger information
- Validate seat-based dynamic forms
- Continue to payment flow

Framework Features

Dynamic Locator Strategy

Parameterized locators implemented for:

- Airlines
- Departure time filters
- Sort options
- Seat-specific passenger forms

Example:

airlineFilter("IndiGo")

departureTimeFilter("6 AM – 12 PM")

sortType("Rating")

firstName("1B")

Benefits

- Reduced duplication
- Improved maintainability
- Easier scalability

Locator Strategy

Priority order:

1. getByRole()

2. getByLabel()

3. getByPlaceholder()

4. getByText()

5. locator()

6. XPath

This approach improves test stability and resilience against UI changes.

Validation Coverage

Search Flow

- Flights tab selection
- Origin and destination selection
- Travel date selection
- Navigation to results page

Flight Results

- Result count validation
- Flight card count validation
- Filtered search result validation

Seat Selection

- Seat selection confirmation
- Dynamic seat verification

Passenger Information

- User details capture
- Dynamic passenger form handling

Logging & Diagnostics

Centralized logging using Winston with:

- Timestamp
- Log Level
- Correlation ID
- Test Name
- Action Message

Example:

Timestamp=2026-07-17T02:34:11.400Z

Level=[INFO]

CID=54919db4

Test=@smoke Flight Booking

Message=Seat 1B selected

``

Security Controls

Secret Management

Sensitive data stored outside source code using:

- .env
- GitHub Secrets

Log Redaction

Automatically masks:

password

token

authorization

email

cardnumber

cvv

Example:

{

"password": "[REDACTED]"

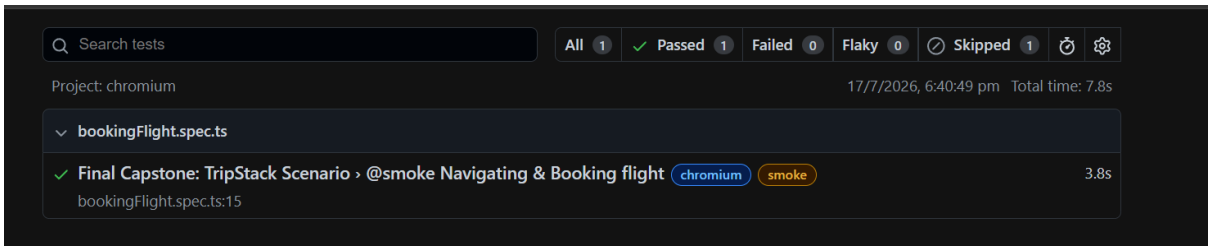
}

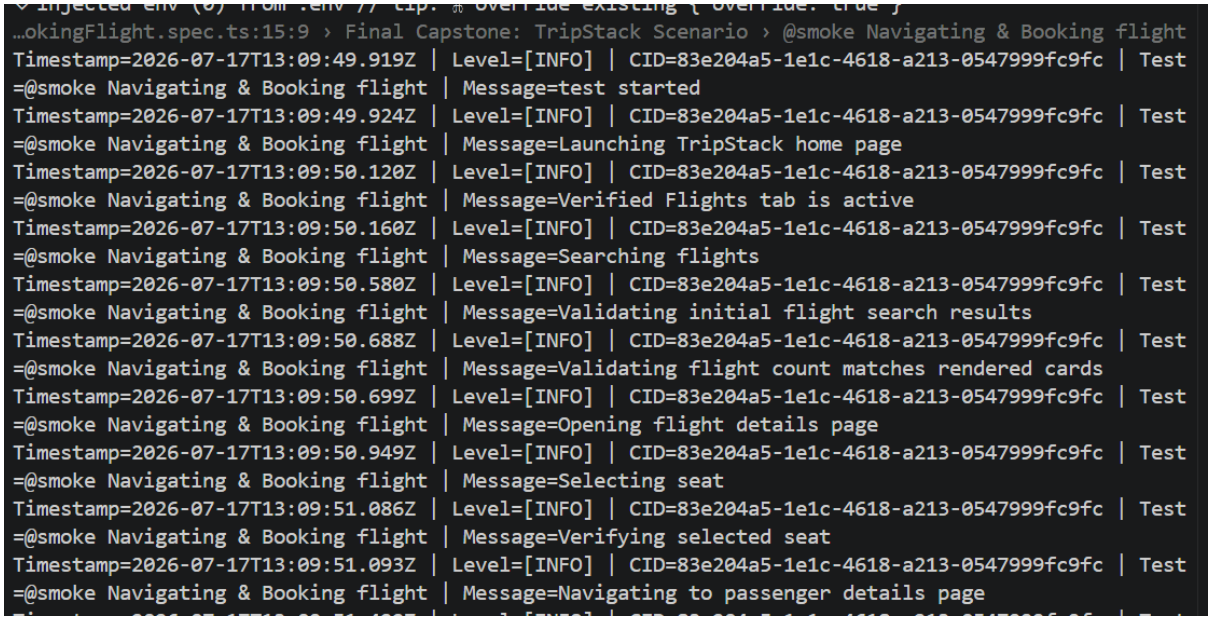
Evidence Collection

Framework supports:

- Structured logs
- Failure screenshots
- Playwright traces
- Test attachments
- Diagnostic artifacts

This enables faster debugging and root cause analysis.





CI/CD Readiness

Implemented for GitHub Actions with support for:

- Environment-based execution
- Secret injection
- Tagged test execution
- Artifact publishing
- Automated test reporting

Key Design Decisions

Flow Layer

Separates business workflows from UI implementation.

Page Object Model

Centralizes locators and actions for easier maintenance.

Dynamic Locators

Reduces hardcoded locators and improves framework scalability.

Evidence-Driven Testing

Captures logs, screenshots, and traces to support failure investigation.

Current Coverage

- ✅ Flight Search
- ✅ Flight Listing Validation
- ✅ Airline Filtering
- ✅ Time Filtering
- ✅ Price Filtering
- ✅ Sorting Validation
- ✅ Seat Selection
- ✅ Passenger Details
- ✅ Structured Logging
- ✅ Secret Redaction
- ✅ Evidence Collection
- ✅ GitHub Secret Integration
- ✅ CI/CD Ready Framework

Overall Assessment

The framework demonstrates enterprise-level automation practices through layered architecture, dynamic locator management, secure configuration handling, structured diagnostics, and CI/CD integration. The solution is maintainable, scalable, and aligned with industry-standard UI automation design principles.

==========
==========
==========

TripStack API Automation Framework Report

Executive Summary

A structured API automation framework was developed for the TripStack flight booking platform using Java, Maven, RestAssured, and JUnit 5. The framework validates critical

booking workflows through automated end-to-end API testing, including authentication, flight search, seat selection, booking lifecycle execution, security validation, and database verification.

The solution follows a modular, maintainable architecture that separates business operations, configuration management, database validations, and test execution logic. The framework is designed to support scalable automation, continuous testing, and future CI/CD integration while improving test reliability and maintainability.

Project Overview

Application

TripStack Travel Booking Platform

Domain

Travel & Transportation

Objective

To automate the validation of the complete flight booking lifecycle through API testing while ensuring data integrity, security compliance, and booking accuracy across application and database layers.

Scope

The automation framework validates:

- User Authentication
- Flight Search
- Seat Availability
- Booking Hold Creation
- Payment Processing
- Booking Confirmation
- Booking State Transitions
- Database Verification
- Security Validation
- Response Time Verification

Technology Stack

Component	Technology
Language	Java
Build Tool	Maven
API Testing Library	RestAssured
Test Framework	JUnit 5
JSON Processing	Jackson
Database Validation	PostgreSQL JDBC
Reporting	Maven Surefire Reports
Configuration	Properties Files
Version Control	Git
CI/CD Ready	GitHub Actions / Jenkins / Azure DevOps

Framework Architecture

Configuration Layer

↓

Base Test Layer

↓

Service Layer

↓

Database Validation Layer

↓

Business Test Layer

Layer Responsibilities

Configuration Layer

- Environment Management

- API Endpoints
- Database Configuration
- User Credentials
- Runtime Parameters

Base Test Layer

- Authentication Setup
- Request Specification
- Shared Test Utilities
- Common Test Initialization

Service Layer

Encapsulates all API operations including:

- Login Service
- Flight Search Service
- Seat Map Service
- Booking Service
- Payment Service

Database Validation Layer

Responsible for:

- Booking Validation
- Data Integrity Checks
- Ownership Verification
- Status Verification

Business Test Layer

Contains business-focused test scenarios and assertions.

Automated Business Flow

Authentication

Objective

Validate secure user authentication.

Steps

1. Login using valid user credentials
2. Retrieve authentication token
3. Initialize clean namespace for execution

Validations

- Successful authentication
- Token generation
- Authorized access verification

Flight Search

Objective

Validate search functionality and flight availability.

Steps

1. Search flights using source and destination
2. Retrieve available flight options
3. Select target flight
4. Retrieve seat map

Validations

- Search response correctness
- Flight availability
- Seat map retrieval
- Response integrity

Booking Lifecycle

Objective

Validate the complete booking flow.

Steps

1. Retrieve available seats
2. Create booking hold
3. Process payment
4. Confirm booking
5. Retrieve booking details

Validations

- Hold creation
- Payment success
- Booking confirmation
- Status progression validation

Database Validation

Objective

Verify successful persistence of booking information.

Validations

- Booking record creation
- Customer ownership mapping
- Booking status correctness
- Payment state verification
- Data consistency checks

Security Validation

Objective

Validate access control and authorization mechanisms.

Covered Scenarios

- Invalid token validation
- Missing token validation
- Unauthorized access attempts

- Authentication failure handling

Validations

- Correct HTTP status codes
- Proper error handling
- Access restriction enforcement

Resilience Validation

Objective

Assess API responsiveness and operational stability.

Validations

- Response time thresholds
- Endpoint stability
- Service availability
- High-level performance verification

Framework Features

Service-Oriented Design

All API interactions are encapsulated within reusable service classes.

Benefits:

- Reduced code duplication
- Improved maintainability
- Easier test expansion

Reusable Test Infrastructure

Common functionality is centralized within base classes and utilities to ensure framework consistency.

Configuration-Driven Execution

Environment-specific values are externalized to configuration files, enabling seamless execution across multiple environments.

Database-Backed Validation

The framework validates API results against backend database records to ensure end-to-end data accuracy and business consistency.

Validation Coverage

Functional Coverage

- ✅ Authentication APIs
- ✅ Flight Search APIs
- ✅ Flight Availability APIs
- ✅ Seat Map APIs
- ✅ Booking Hold APIs
- ✅ Payment APIs
- ✅ Booking Confirmation APIs
- ✅ Booking Retrieval APIs

Security Coverage

- ✅ Invalid Authentication Testing
- ✅ Unauthorized Access Validation
- ✅ Token Handling Verification
- ✅ Error Response Verification

Database Coverage

- ✅ Booking Record Validation
- ✅ Ownership Verification

✅ Status Verification

✅ Data Integrity Validation

✅ State Transition Verification

Resilience Coverage

✅ Response Time Validation

✅ Service Availability Checks

✅ Workflow Stability Validation

Reporting and Logging

Reporting

Execution results are generated through Maven test execution reports and assertion outcomes.

Captured Information

- Test Execution Results
- Pass / Fail Status
- Assertion Details
- Failure Diagnostics

Logging

The framework provides structured execution logging covering:

- Request Payloads
- Response Payloads
- Authentication Status
- Booking Lifecycle Events
- Database Validation Results
- Response Times
- Exception Details

This supports efficient debugging and root cause analysis.

Security Controls

Secret Management

Sensitive data is managed outside source code through configuration-driven approaches.

Protected Assets

- User Credentials
- Authentication Tokens
- Database Credentials
- Environment Endpoints

Data Protection

The framework avoids exposing sensitive runtime information within test outputs and logs.

Evidence Collection

The framework generates execution evidence through:

- Maven Execution Logs
- Assertion Results
- API Responses
- Database Validation Results
- Status Verification Outputs

These artifacts support auditability and defect investigation.

CI/CD Readiness

The framework is fully compatible with modern CI/CD platforms.

Supported Platforms

- GitHub Actions

- Jenkins
- Azure DevOps

CI/CD Features

- Scheduled Execution
- Regression Execution
- Environment-Based Deployment
- Automated Reporting
- Artifact Generation

Benefits

- Faster Feedback Cycles
- Continuous Quality Validation
- Reduced Manual Testing Effort
- Improved Release Confidence

Key Design Decisions

Modular Architecture

Business logic, API interactions, and validations are separated into dedicated layers.

Service Abstraction

API operations are centralized within reusable service classes.

Configuration Externalization

Environment-specific settings are managed externally to improve portability and maintainability.

Database Verification

Backend validation strengthens confidence in booking lifecycle correctness and data integrity.

Security-First Approach

Authentication and authorization scenarios are included as part of the automated regression suite.

Current Coverage

- ✅ Authentication Flow
- ✅ Flight Search
- ✅ Flight Availability Validation
- ✅ Seat Map Retrieval
- ✅ Booking Hold Creation
- ✅ Payment Processing
- ✅ Booking Confirmation
- ✅ Booking Retrieval
- ✅ Database Verification
- ✅ Security Validation
- ✅ Response Time Validation
- ✅ End-to-End Booking Workflow Validation

Key Achievements

- ✅ End-to-End API Workflow Automation
- ✅ Reusable Service-Based Framework Design
- ✅ Database-Backed Validation Strategy
- ✅ Secure Configuration Management
- ✅ Automated Security Testing
- ✅ Response Time Validation
- ✅ Maintainable Test Architecture
- ✅ CI/CD Ready Implementation
- ✅ Scalable Automation Foundation
- ✅ Improved Regression Coverage

Overall Assessment

The TripStack API Automation Framework successfully validates the complete booking lifecycle from authentication through booking confirmation and verification. The framework employs a modular architecture, reusable service abstraction, database validation capabilities, security testing practices, and configuration-driven execution aligned with industry-standard API automation frameworks.

The solution provides a scalable and maintainable foundation for continuous testing, regression execution, CI/CD adoption, and future expansion while ensuring functional correctness, data integrity, and operational reliability across the TripStack booking ecosystem.